Introduction to Artificial Intelligence

Lecture 4 · Local search and genetic algorithms

Hill climbing · restarts · populations · evolution

Today

01	**When the path does not matter**	Optimization, local search, the landscape
02	**Hill climbing**	Steepest ascent on 8-queens, and where it gets stuck
03	**Escaping local optima**	Sideways moves, restarts, min-conflicts, the TSP
04	**Genetic algorithms**	Populations, crossover and mutation
05	**Designing a genetic algorithm**	Selection, hyperparameters, exploration

01

When the path does not matter

So far, the answer was a path

In the last three lectures, a solution was a sequence of actions: the moves of the jugs, of the 8-puzzle, the roads of a route. Search built paths, step by step.

In many problems, only the final configuration matters:

- **8-queens**: where the queens stand, not the order in which they were placed.
- **Timetables**: which course goes in which room and slot.
- **Circuit layout, portfolio selection, the design of a wing**: a configuration to choose, with a quality to optimize.

Or the path is part of the candidate

- **The traveling salesman problem**: visit every city once and return, as cheaply as possible. A tour *is* a path, but it is a complete candidate: an ordering of the cities.
- **A plan** can be a candidate too: a whole list of actions, judged by where it leads.
- Either way, we search among **complete candidates**, and improve them, instead of extending partial paths.

> A *complete-state formulation*: every state is a full candidate solution, good or bad.

Optimization

Each candidate has a value to maximize, or a cost to minimize: an *objective function*.

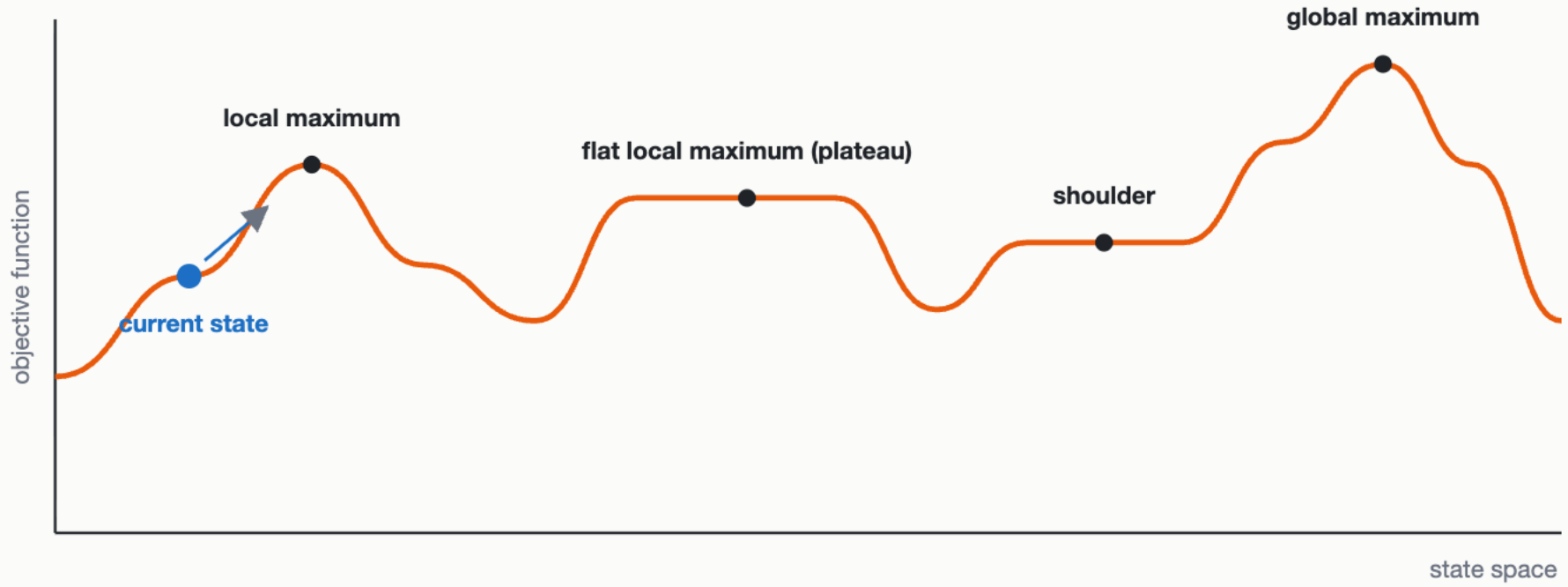
Problem	A candidate	Its cost
8-queens	one queen in each column	pairs of queens attacking each other: 0 is a solution
Traveling salesman	an ordering of the cities	the length of the tour

Often the space is far too large to search systematically, and “very good, quickly” is worth more than “optimal, eventually”.

Local search

- Keep a **single current state**, and move to one of its *neighbors*: candidates that differ by a small change, such as moving one queen.
- Remember no paths and no frontier: **constant memory**, so it works in huge, or even continuous, spaces.
- Guided by the objective function, it often reaches a good solution in very few steps.
- **But it is not systematic**: it may miss the optimum, and it may not tell when it has found it.

The state-space landscape



Local search walks on this landscape, seeing only the neighbors of where it stands. Maximizing value means climbing; minimizing cost, descending.

02

Hill climbing

Hill climbing

```
function HILL-CLIMBING(start):
    current ← start
    loop:
        best ← a neighbor of current with the least cost
        if cost(best) ≥ cost(current): return current
        current ← best
```

- Move to the best neighbor, as long as it improves: *steepest ascent* (here, steepest descent of the cost).
- Like climbing a mountain in thick fog: you only feel the slope under your feet.
- When several neighbors are equally good, choose one at random.

8-queens as local search

12	13	13	13	16	19	13	Q
13	13	12	14	16	18	Q	14
11	13	11	12	16	23	13	13
14	13	13	13	22	Q	13	13
12	15	12	Q	17	18	14	14
13	13	Q	14	17	18	14	14
12	Q	13	15	16	20	13	13
Q	15	14	14	Q	19	15	14

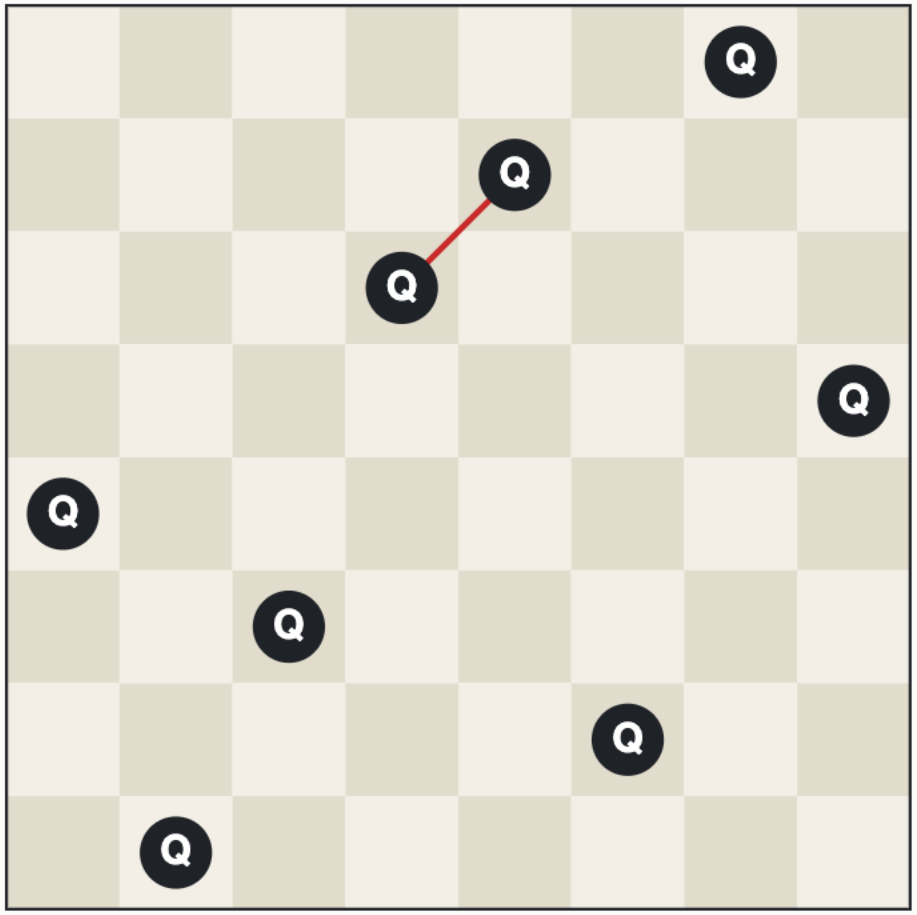
- **State**: one queen per column, $8^8 \approx 16.8$ million boards.
- **Neighbors**: move one queen within its column: $8 \times 7 = 56$.
- **Cost h**: the pairs attacking each other. Here, h = 17.
- Each number: h after moving that column's queen there. The best moves give 11.

How well does it work?

Steepest hill climbing on 8-queens	
Random boards solved	14%
Moves, when it solves the board	4
Moves, when it gets stuck	3

Measured on 2,000 random boards. Very fast: about 4 moves among 16.8 million boards. But it gets stuck 86% of the time.

Where hill climbing gets stuck



- **Local minimum**: on this board one pair attacks (h = 1), and every one of the 56 moves makes things worse.
- **Plateau**: neighbors as good as the current state; no direction to follow.
- **Ridge**: a sequence of local optima, too steep for single moves.

Hill climbing in Python

```
def best_neighbor(state: S, neighbors: Callable[[S], list[S]],
                  cost: Callable[[S], float]) → S:
    candidates = neighbors(state)
    costs = [cost(c) for c in candidates]
    least = min(costs)
    return random.choice([c for c, k in zip(candidates, costs) if k == least])

def hill_climbing(start: S, neighbors: Callable[[S], list[S]],
                  cost: Callable[[S], float]) → S:
    current = start
    while True:
        best = best_neighbor(current, neighbors, cost)
        if cost(best) ≥ cost(current):
            return current
        current = best

board = hill_climbing(random_board(8), neighbors, attacking_pairs)
```

Generic: any problem, given its neighbors and its cost as functions.

03

Escaping local optima

Sideways moves

- Accept a move to an *equal* neighbor, to cross plateaus and shoulders.
- Limit the number of sideways moves in a row (say 100), or the search may wander forever on a flat local optimum.

8-queens	Steepest	With sideways moves
Boards solved	14%	94%
Moves when it succeeds	4	19
Moves when it fails	3	64

Stochastic and first-choice hill climbing

- **Stochastic hill climbing**: choose at random among the improving moves, possibly favoring the steeper ones.
- **First-choice hill climbing**: generate random neighbors one at a time, and take the first one that improves.
- Useful when a state has thousands of neighbors: there is no need to evaluate them all.
- On 8-queens, first choice solves 11% of the boards, in 6 moves on average: close to steepest ascent, with far fewer evaluations per move.

Random-restart hill climbing

“If at first you don’t succeed, try, try again”: run hill climbing from new random states until one run succeeds.

- If each run succeeds with probability p, we expect 1/p runs: for 8-queens, 1/0.14 ≈ 7.
- Measured: **7.0 runs and 22 moves in total**, on average. It always ends up solving the problem.
- Complete with probability approaching 1, given enough restarts.

Comparing the variants

On 8-queens	Boards solved	Moves
Steepest hill climbing	14%	4
With up to 100 sideways moves	94%	19
First-choice hill climbing	11%	6
Random restarts of steepest hill climbing	100%	22 in total

Averages over 2,000 random boards; the moves are those of the successful runs. Simple ideas, big effects: sideways moves and restarts turn 14% into nearly 100%.

Min-conflicts: large boards, few steps

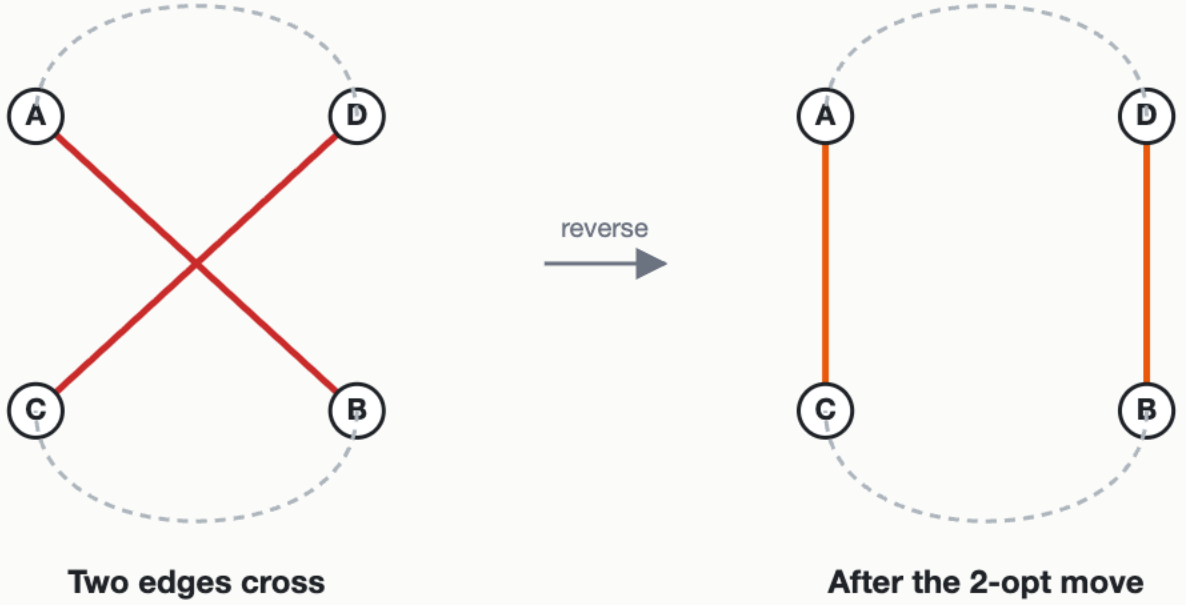
```
function MIN-CONFLICTS(board, max-steps):
    for step from 1 to max-steps:
        if no queen is attacked: return board
        Q ← a random queen under attack
        move Q to a row of its column with the fewest attacks
    return failure
```

N queens	Steps	Time, in Python
100	133	0.003 s
1,000	757	0.2 s
10,000	6,214	17 s

The heuristic leads almost straight to a solution: nearly every step removes a conflict.

Local search for the traveling salesman

- **State**: a tour of n cities. With 30 cities, 29!/2 ≈ 4.4 × 10^{30} different tours.
- **Neighbors**: *2-opt* moves, which reverse a segment of the tour: about 400 for 30 cities.
- A 2-opt move removes two edges and reconnects the tour; it can uncross two edges.

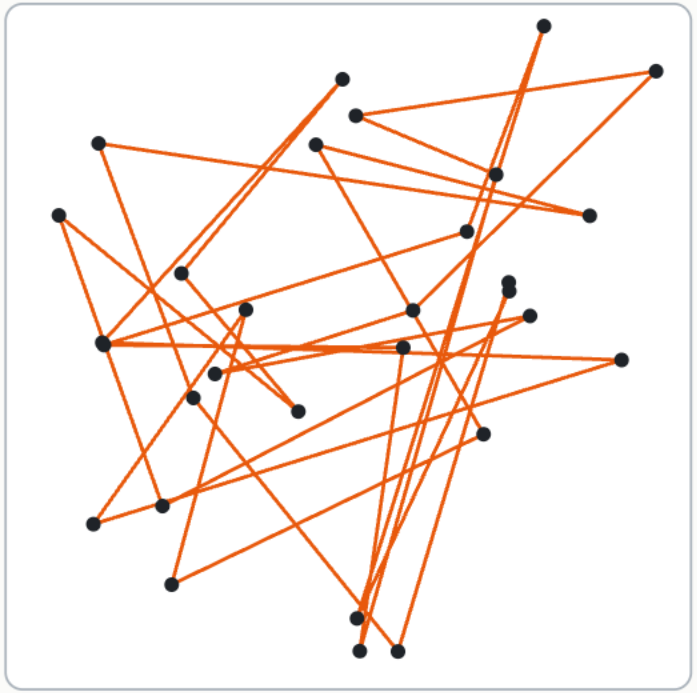


Two edges cross

After the 2-opt move

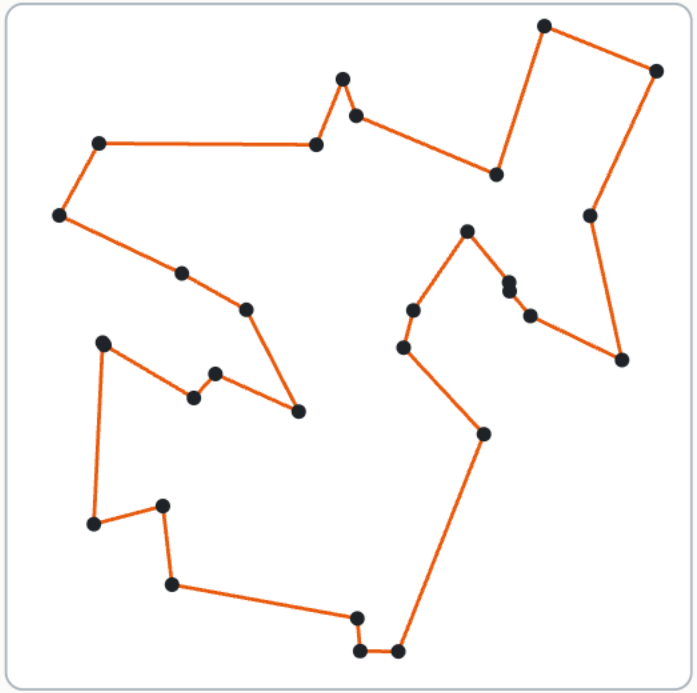
2-opt hill climbing on 30 cities

A random tour



length 15.44

After 2-opt hill climbing



length 4.65, 24 moves

From 15.44 to 4.65 in 24 moves, in 0.03 s. Random tours average 14.69, and the nearest-neighbor heuristic gives 5.53. Thirty restarts end between 4.55 and 5.19.

Near misses

How close does local search get? On 12 cities the exact optimum can be computed (by dynamic programming), and compared with 200 runs of 2-opt hill climbing from random tours:

2-opt hill climbing, 12 cities	
Runs that find the optimal tour	70%
Length above the optimum, on average	1.3%
Length above the optimum, at worst	6%

Local search rarely proves anything, but it gets very close, very fast.

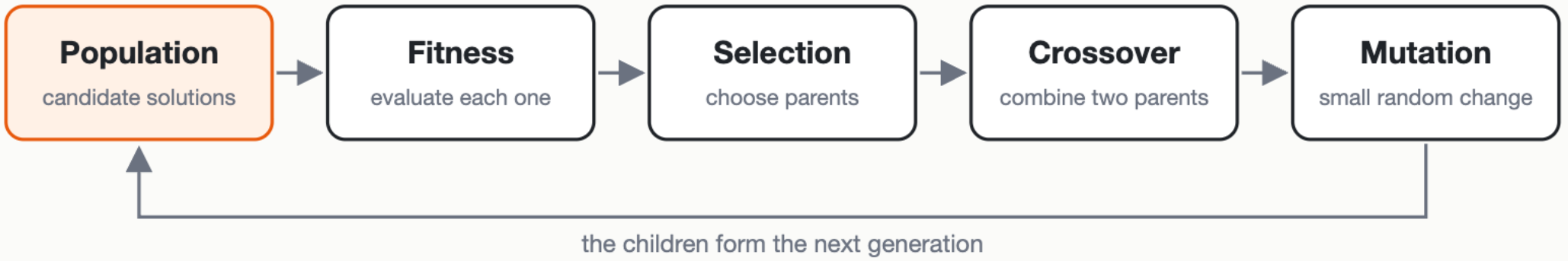
04

Genetic algorithms

From one state to many: local beam search

- Keep **k states** instead of one. At each step, generate the neighbors of all of them, and keep the k best.
- Not k independent restarts: the searches share information, as good states attract the others.
- The risk: all k states crowd into the same region.
- *Stochastic* beam search keeps k states chosen at random, the better ones more likely: a kind of natural selection.

Evolution as search



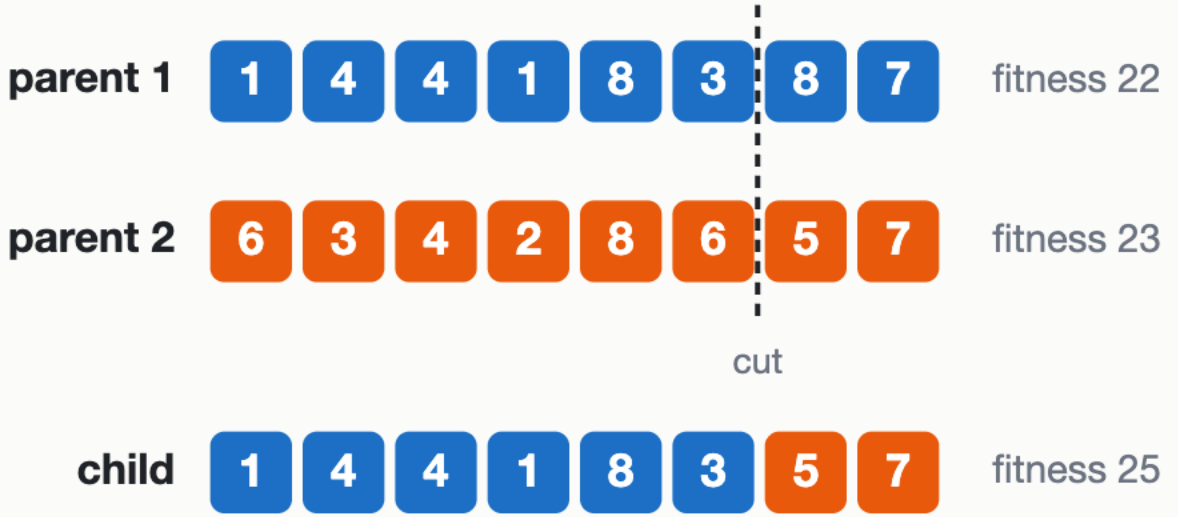
- A **population** of candidates, called individuals, encoded as strings.
- Fitter individuals are more likely to become parents.
- Children combine their parents (**crossover**), with small random changes (**mutation**).

Encoding 8-queens

- An individual: 8 digits, the row of the queen in each column, as in hill climbing.
- **Fitness**: the pairs of queens that do *not* attack each other, out of 28. A solution has fitness 28; fitness is 28 − h.
- An initial population: random boards.

> Choosing the encoding is the main design decision: crossover must be able to combine useful parts of two parents.

Crossover



- **One-point crossover**: cut at a random position; the child takes the left part of one parent and the right part of the other.
- Good parts of different parents can combine: here, parents of fitness 22 and 23 give a child of 25.

Mutation

- With some probability, the *mutation rate*, a child gets a small random change: here, one queen moved to a random row.
- Crossover only recombines what the population already has. If no board has a queen in row 3 of column 5, crossover can never create one; mutation can.
- Mutation keeps **diversity**, so the search does not stall.

```
def one_point_crossover(parent1: list[int], parent2: list[int]) → list[int]:
    cut = random.randrange(1, len(parent1))
    return parent1[:cut] + parent2[cut:]


def mutate_queens(board: list[int]) → list[int]:
    child = board.copy()
    child[random.randrange(len(child))] = random.randrange(len(child))
    return child
```

A genetic algorithm in Python

```
def genetic_algorithm(population: list[I], fitness: Callable[[I], float],
                      select: Selection[I], crossover: Callable[[I, I], I],
                      mutate: Callable[[I], I], mutation_rate: float,
                      generations: int) -> I:
    for _ in range(generations):
        scores = [fitness(individual) for individual in population]
        best = population[scores.index(max(scores))]
        children = [best]                          # elitism: the best survives
        while len(children) < len(population):
            child = crossover(select(population, scores), select(population, scores))
            if random.random() < mutation_rate:
                child = mutate(child)
            children.append(child)
        population = children
    return max(population, key=fitness)
```

Generic again: the problem provides the fitness, the crossover and the mutation; the selection scheme is a parameter.

A genetic algorithm on 8-queens

8-queens, at most 500 generations	Genetic algorithm	Hill climbing with restarts
Runs that find a solution	62%	100%
Evaluations of the cost or fitness	about 3,600	about 1,650

Population of 100, tournaments of 3, mutation rate 0.2, the best kept; 50 runs.

Here the genetic algorithm loses: crossover of 8-queens boards rarely combines useful pieces. Genetic algorithms pay off when it does.

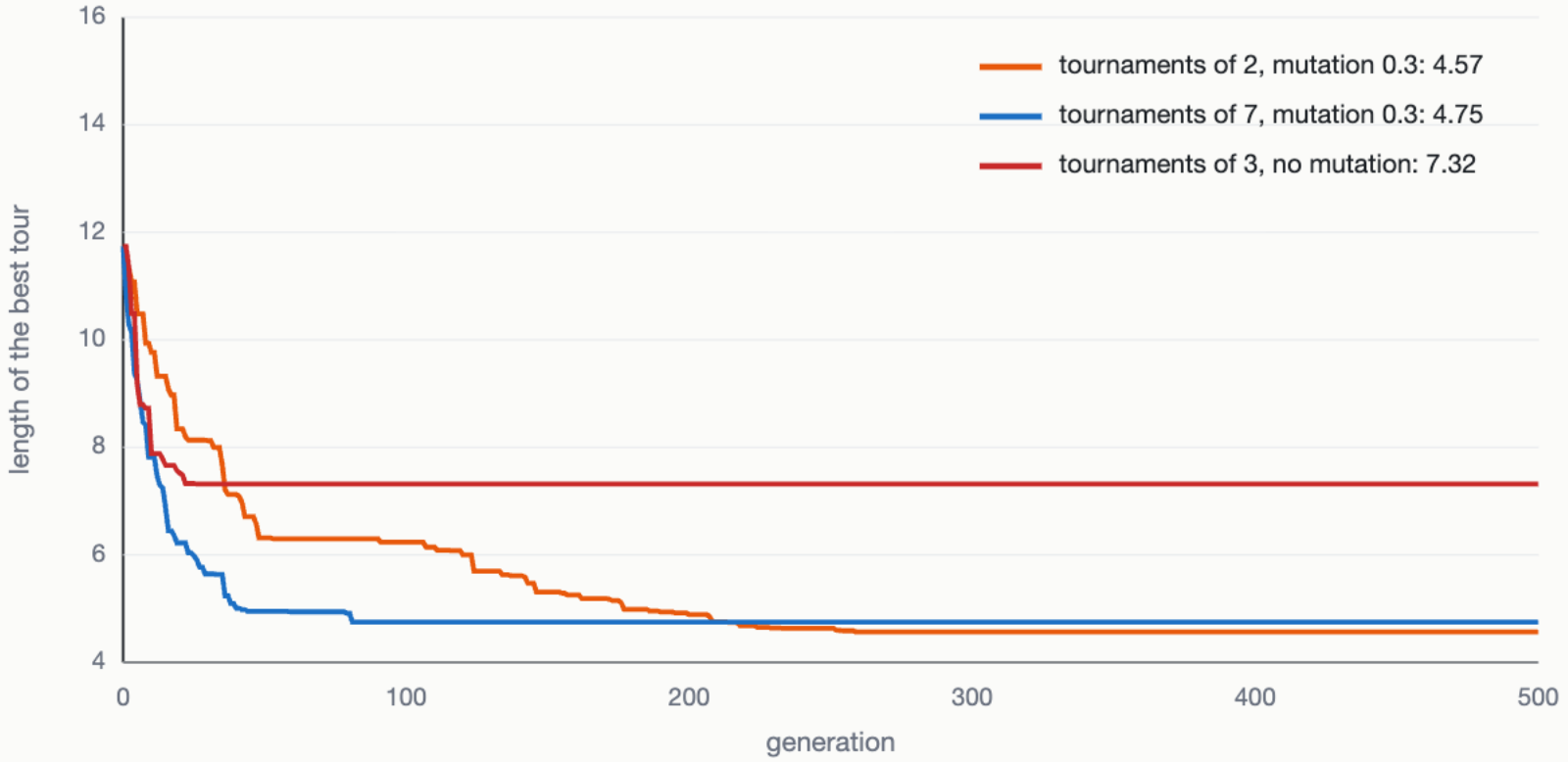
A genetic algorithm for the TSP

- One-point crossover of two tours repeats some cities and drops others: the child is not a tour.
- **Order crossover**: keep a segment of parent 1 in place, and fill the other positions with the remaining cities, in the order they appear in parent 2.



Mutation: reverse a random segment, a random 2-opt move.

Evolving tours



30 cities, population 100. Without mutation, the population stops improving after about 40 generations.

05

Designing a genetic algorithm

Selection

Scheme	How parents are chosen
Fitness-proportionate (roulette)	with probability proportional to fitness; needs positive fitness, and is sensitive to its scale
Tournament	pick k individuals at random, keep the fittest; k sets the pressure
Rank	by position in the ranking, ignoring the size of the differences
Elitism	the best few pass to the next generation unchanged

```
def tournament(population: list[I], scores: list[float], k: int = 3) → I:
    contestants = random.sample(range(len(population)), k)
    return population[max(contestants, key=lambda i: scores[i])]
```

Selection pressure

- **Strong pressure** (large tournaments): fast progress, but the population soon looks alike, and gets stuck: *premature convergence*.
- **Weak pressure**: slower, but it keeps exploring.

TSP, 30 cities, 5 runs each	Best tour, on average
Tournaments of 2	4.62
Tournaments of 3	4.67
Tournaments of 7	4.73
Tournaments of 3, no mutation	6.07

Hyperparameters

Population	Solved	Evaluations
20	42%	2,700
50	50%	3,200
100	62%	3,600
200	78%	6,400

Mutation rate	Solved
0	18%
0.05	52%
0.2	62%
0.5	88%
1	94%

8-queens, 50 runs per setting, at most 500 generations; the other settings as before. Larger populations solve more, but cost more evaluations; here mutation matters a lot.

Exploration and exploitation

- **Exploitation**: selection and crossover concentrate the search around the best individuals found.
- **Exploration**: mutation and large populations keep looking elsewhere.
- Too much exploitation: premature convergence. Too much exploration: a random search.
- The hyperparameters interact, and depend on the problem: tune them by experiment, over several runs, since every run is random.

When are genetic algorithms worth it?

- When an encoding lets crossover combine meaningful parts: building blocks that are good on their own.
- In large, rugged spaces where no good neighborhood is known.
- When evaluations can run in parallel: a whole population at once.
- But they have many hyperparameters and no guarantees, and simpler local search is often as good: try hill climbing with restarts first.

Summary

- When only the final configuration matters, search among complete candidates and improve them.
- Hill climbing follows the heuristic and gets close very quickly, but stops at local optima.
- Sideways moves, random restarts and randomness escape them; min-conflicts solves huge N-queens boards in seconds.
- Genetic algorithms evolve a population, with selection, crossover and mutation.
- Their encoding, selection scheme and hyperparameters decide whether they work.

In the lab

1. Neighbors for 8-queens, then steepest hill climbing: how often does it solve the problem?
2. First-choice hill climbing and random restarts.
3. The TSP: 2-opt neighbors, and hill climbing, drawn as an image.
4. A genetic algorithm for 8-queens: tournament selection, crossover and mutation.
5. If you have time: order crossover, a genetic algorithm for the TSP, and its hyperparameters.

Next: constraint satisfaction

- Problems described by variables, their domains and constraints: N-queens, Sudoku, timetables, map coloring.
- Backtracking search with inference, and min-conflicts again.

Reading: Russell and Norvig, *Artificial Intelligence: A Modern Approach*, 4th edition, section 4.1.